

Mochila Cheia — Guia da Equipe (EP2 · PI3)

Como integrar o MVP Web · Divisão de tarefas · Roteiro do vídeo

Projeto Integrado III [ADS0038] — Prof. Luís Fabrício de Freitas Souza

1. Para que serve este guia

Este documento é o **manual de bordo** da equipe para fechar o Entregável Parcial 2 (EP2) do PI3 — o **Modelo Arquitetural do MVP Web** do Mochila Cheia.

O **esqueleto da aplicação já está pronto e funcionando** (parte do Rodrigo). O que falta é cada pessoa preencher o seu módulo seguindo o padrão já estabelecido. Aqui está: o que já existe, como rodar, **como integrar seu pedaço ao sistema**, a divisão de tarefas com checklists e o **roteiro do vídeo** de apresentação.

Regra de ouro: ninguém inventa estrutura nova. Todo mundo copia o padrão da `home` (que já funciona) e adapta. Isso evita conflito e mantém a arquitetura consistente — que é exatamente o que está sendo avaliado.

2. O que já está pronto (não mexer, só usar de exemplo)

Item	Arquivo	Status
Application Factory	src/web/__init__.py	✓ Pronto
Configuração	src/web/config.py	✓ Pronto
Camada de persistência (conexão + init-db)	src/web/database.py	✓ Pronto
Padrão Repository (base)	src/web/repositories/base_repository.py	✓ Pronto
Exemplo completo (home + estatísticas)	src/web/blueprints/home.py + repositories/estatisticas_repository.py	✓ Pronto
Layout base + bottom tab bar	src/web/templates/base.html	✓ Pronto
CSS responsivo base	src/web/static/css/style.css	✓ Pronto
Domínio POO (regras de negócio)	src/models/*	✓ Reaproveitado
Banco físico + dados de exemplo	database/schema.sql + seed.sql	✓ Pronto
Documento arquitetural + diagramas	docs/EP2-PI3_Arquitetura.md + docs/diagramas/	✓ Pronto

Os demais módulos (auth, itens, solicitações, moderação, mensagens, notificações) estão como **stubs**: a rota existe e abre uma tela de “em desenvolvimento” que mostra o responsável. **Seu trabalho é transformar o stub em módulo real.**

3. Como rodar o projeto (todos)

```
# 1. Clonar e entrar na pasta
git clone https://github.com/chico-piaba/Mochila-Cheia.git
cd "Mochila Cheia - Projeto Integrado"

# 2. Criar ambiente virtual e instalar dependências
python3 -m venv .venv
source .venv/bin/activate      # Windows: .venv\Scripts\activate
pip install -r requirements.txt

# 3. Criar o banco com dados de exemplo
flask --app run init-db

# 4. Subir o servidor
flask --app run run --debug
# Abrir no navegador: http://127.0.0.1:5000
```

Se a home aparecer com os cards de estatística e a lista de itens, está tudo certo.

4. A arquitetura em 1 minuto

O sistema é **MVC em 4 camadas**. Uma requisição sempre desce e sobe por elas:

```
Navegador
| (HTTP)
1) APRESENTAÇÃO → templates/*.html + static/css (o que o usuário vê)
|
2) CONTROLE     → blueprints/*.py              (recebe a rota, coordena)
|
3) DOMÍNIO      → src/models/*.py              (regras de negócio, P00)
|
4) PERSISTÊNCIA → repositories/*.py + database.py (fala com o SQLite)
|
SQLite (mochila_cheia.db)
```

Cada camada só conversa com a vizinha. O blueprint nunca escreve SQL direto; ele chama um repository. O template nunca acessa o banco; ele recebe dados prontos do blueprint.

5. Como integrar seu módulo ao sistema (passo a passo)

Use a **home como molde**. Para criar qualquer módulo, são 3 passos:

Passo 1 — Criar o Repository (camada de dados)

Crie `src/web/repositories/<nome>_repository.py`, herdando de `BaseRepository`:

```

from src.web.repositories.base_repository import BaseRepository

class ItemRepository(BaseRepository):
    tabela = "ITEM"
    chave = "id_item"

    def disponiveis(self, categoria=None):
        sql = "SELECT * FROM vw_itens_disponiveis"
        params = ()
        if categoria:
            sql += " WHERE categoria = ?"
            params = (categoria,)
        return self._conn().execute(sql, params).fetchall()

    def criar(self, titulo, descricao, ...):
        sql = "INSERT INTO ITEM (titulo, descricao, ...) VALUES (?, ?, ...)"
        cur = self._conn().execute(sql, (titulo, descricao, ...))
        self._conn().commit()
        return cur.lastrowid

```

Reproveite as **views** que já existem no `schema.sql` (`vw_itens_disponiveis`, `vw_solicitacoes_pendentes`, `vw_estatisticas`) — elas já fazem os JOINS por você.

Passo 2 — Implementar o Blueprint (camada de controle)

Abra o stub (ex.: `src/web/blueprints/itens.py`) e substitua o conteúdo do `render_template("placeholder.html", ...)` pela lógica real:

```

from flask import Blueprint, render_template, request, redirect, url_for
from src.web.repositories.item_repository import ItemRepository

itens_bp = Blueprint("itens", __name__)

@itens_bp.route("/")
def listar():
    repo = ItemRepository()
    categoria = request.args.get("categoria")
    itens = repo.disponiveis(categoria=categoria)
    return render_template("itens/listar.html", itens=itens)

```

O blueprint **já está registrado** em `src/web/blueprints/__init__.py` — não precisa mexer nisso.

Passo 3 — Criar os Templates (camada de apresentação)

Crie `src/web/templates/itens/listar.html` **estendendo a base** (herda o cabeçalho e a bottom tab bar):

```
{% extends "base.html" %}
{% block titulo %}Buscar itens{% endblock %}
{% block conteudo %}
    <h1>Itens disponíveis</h1>
    <ul class="grade-itens">
        {% for item in itens %}
            <li class="card-item"><h3>{{ item.titulo }}</h3>...</li>
        {% endfor %}
    </ul>
{% endblock %}
```

Pronto. Rode `flask --app run run --debug`, acesse a rota e teste. Se funcionar como a home, está integrado corretamente.

Regras para não quebrar o sistema

- **Domínio:** as regras de negócio (ex.: aprovar item, reservar) já estão nas classes em `src/models/`. **Use os métodos delas** (`item.aprovar()`, `solicitacao.aceitar()`), não reimplamente.
- **Senhas:** no cadastro/login, use a classe `Usuario` (ela já faz o hash). Nunca salve senha em texto plano.
- **Sessão:** para “usuário logado”, use `session["usuario_id"]` do Flask.
- **Commit no banco:** toda escrita (INSERT/UPDATE) precisa de `self._conn().commit()` no repository.

6. Divisão de tarefas (com checklist)

A divisão segue as funções definidas na Unidade 1. Cada um trabalha no seu módulo, em paralelo, sem conflito.

Rodrigo Lima Diôgo — Gestão, Comunicação e Arquitetura

- ☒ Definir o modelo arquitetural (4 camadas / MVC)
- ☒ Montar o esqueleto: app factory, persistência, repositories, home de exemplo
- ☒ Escrever o documento arquitetural + diagramas (`docs/EP2-PI3_Arquitetura.md`)
- ☒ Subir tudo no GitHub e escrever este guia
- ☐ Coordenar a integração dos módulos e revisar os PRs/commits da equipe
- ☐ Montar o vídeo final (edição) a partir das falas de todos

Júlio César Batista da Silva — Backend e Análise de Fluxo

- ☐ `repositories/usuario_repository.py` (criar/buscar por email)
- ☐ **Módulo auth** (`blueprints/auth.py`): cadastro, login, logout (usar `Usuario` + `session`)
- ☐ `repositories/item_repository.py` + **módulo itens** (publicar, buscar, detalhe)

- ☐ **Módulo solicitações** (solicitar / aceitar / finalizar) usando `Solicitacao`
- ☐ **Módulo moderação** (fila + aprovar/recusar) usando `Item.aprovar()/recusar()`
- ☐ Proteger rotas de moderador (verificar tipo do usuário na sessão)

■ Francisco Robson Paulino Cruz — IHC/UX

- ☐ Traduzir as telas do **Figma** para os templates Jinja (estendendo `base.html`)
- ☐ Refinar o `static/css/style.css` com a identidade visual do protótipo (paleta, tipografia)
- ☐ **Componente Extensionista:** escrever no `README.md` a seção “**O que é Arquitetura de Software?**” (texto autoral — não copiar da internet)
- ☐ Garantir acessibilidade (contraste $\geq 4.5:1$, alvos de toque $\geq 48\text{px}$, textos alternativos)

■ Lucas do Nascimento Souza — IHC/UX

- ☐ **Módulo mensagens/chat** (frontend): lista de conversas + tela de conversa
- ☐ **Módulo notificações** (frontend): lista de alertas
- ☐ Revisar a navegação (bottom tab bar) e a jornada dos 3 perfis
- ☐ Telas de feedback: confirmações, mensagens de erro, estados vazios

■ Maria da Conceição Freitas Lopes — Documentação e IHC/UX

- ☐ Consolidar e revisar o `docs/EP2-PI3_Arquitetura.md` (revisão textual)
- ☐ Organizar as **evidências** (prints das telas rodando, fotos/prints de reuniões)
- ☐ Preencher os campos `[INSERIR ...]` do relatório (links, evidências)
- ☐ Conferir o checklist de entrega (seção 8)

7. Convenções (Git e código)

Git — fluxo simples para 5 pessoas:

- Antes de começar o dia: `git pull` (sempre puxar o que os outros subiram).
- Commits pequenos e descritivos, em português. Ex.: `feat: módulo de itens (listagem e busca)`.
- Cada um mexe **principalmente no seu módulo** → quase nunca dá conflito.
- Se der conflito, chama o Rodrigo (coordenação) antes de forçar qualquer coisa.

Código:

- Seguir o estilo do que já existe: docstring no topo do arquivo dizendo o que o módulo faz e quem é o responsável.
- Nomes em português, claros. Funções curtas.
- Nada de SQL dentro de blueprint; nada de acesso a banco dentro de template.

8. Checklist de entrega do EP2 (PI3)

A entrega são **2 arquivos no AVA + repositório atualizado**:

- ☐ **Arquivo 1 — Relatório:** docs/EP2-PI3_Arquitetura.md exportado em PDF, com vídeo e evidências preenchidos
- ☐ **Arquivo 2 — Documentação arquitetural / diagramas:** já contemplado no relatório + docs/diagramas/
- ☐ **README** atualizado com a seção “O que é Arquitetura de Software?” (Robson)
- ☐ **Repositório** organizado, funcional e com link no relatório
- ☐ **Diagramas** legíveis e identificados (camadas, componentes, fluxo)
- ☐ **Vídeo** (≤ 5 min) com todos os membros, demonstrando o MVP e a arquitetura
- ☐ Cada membro respondeu o **formulário de autoavaliação** (após enviar)

Critérios avaliados (0–2,0 cada): clareza do modelo · qualidade das justificativas técnicas · uso de padrões e boas práticas · profundidade do README · organização do repositório.

9. Roteiro do vídeo de apresentação (máx. 5 minutos)

O vídeo deve mostrar o **MVP rodando** e explicar a **arquitetura**. **Todos falam**. Sugestão de divisão por tempo:

Tempo	Quem	O que falar / mostrar
0:00–0:30	Rodrigo	Abertura: apresenta a equipe, o problema (custo do material escolar / evasão) e o objetivo do MVP Mochila Cheia.
0:30–1:30	Rodrigo	Arquitetura: mostra o diagrama das 4 camadas (MVC). Explica que o domínio POO foi reaproveitado e por que escolhemos Flask + SQLite. Mostra a estrutura de pastas <code>src/web/</code> .
1:30–2:30	Júlio	Backend ao vivo: sobe o servidor, mostra a home com dados reais. Explica o fluxo de uma requisição (rota → repository → banco → template) e o fluxo de doação (pendente → disponível → reservado → doado).
2:30–3:30	Robson	Interface/IHC: mostra as telas vindas do Figma, a navegação por bottom tab bar, e fala das boas práticas de usabilidade e acessibilidade aplicadas (WCAG, contraste, alvos de toque).
3:30–4:15	Lucas	Mostra a jornada do usuário (doador e receptor), as telas de chat/notificações e como a navegação conecta os 3 perfis.
4:15–4:45	Maria	Mostra a documentação: o documento arquitetural, os diagramas e a seção extensionista “O que é Arquitetura de Software?” no README. Comenta as decisões e boas práticas.
4:45–5:00	Rodrigo	Fechamento: recapitula os padrões usados (MVC, camadas, Repository, componentização) e o impacto social do projeto. Mostra o repositório no GitHub.

Dicas de gravação:

- Cada um grava sua parte (pode ser tela + voz). Rodrigo junta tudo na edição.
- Ter o servidor já rodando e o banco populado (`init-db`) **antes** de gravar.
- Deixar o diagrama de camadas e o documento arquitetural abertos para mostrar na tela.
- Falar olhando para o objetivo: “como a arquitetura ajuda na qualidade, manutenção e escalabilidade”.

Resumo: o esqueleto está pronto e funcionando. Cada um pega seu módulo na seção 6, segue o passo a passo da seção 5 (copiando o padrão da home), e marca o checklist. Dúvidas de integração → Rodrigo. Bora fechar essa entrega! 📦